ASSIGNMENT13.1

2303A51836

BATCH:27

TASK1:

Prompt:

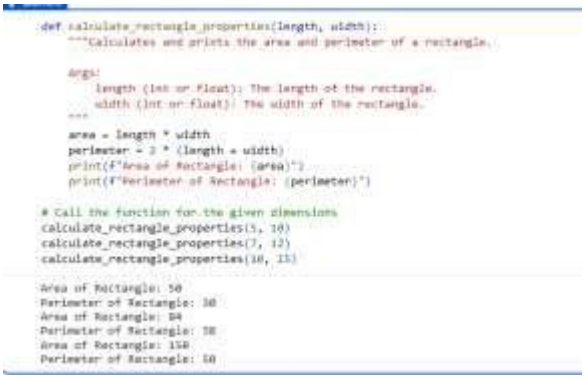
UseAItorefactorthegivenpythonscriptthatcontainsmultiplerepeatedcodeblocks Before

Code Refactoring:

```
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

AfterCodeRefactoring:



TASK2:

Prompt:

UseAItoanalyzeaPythonscriptwithnestedloopsandcomplexconditionals.

Before Code Refactoring:

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
   found = False
for n in names:
   if s == n:
      found = True
   if found:
      print(f"{s} is in the list")
   else:
      print(f"{s} is not in the list")
```

```
Bob is not in the list
Bob is in the list
Bob is in the list
Bob is in the list
```

AfterCodeRefactoring:

```
import time

names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

print("--- Original (Corrected) Code ---")
start_time_original = time.time()
# Corrected legacy inefficient code
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
            break # Exit inner loop once found
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
end_time_original = time.time()
original_duration = end_time_original - start_time_original

print("\n--- Optimized Code (using set) ---")
start_time_optimized = time.time()
# Optimized code using set for faster lookups
names_set = set(names) # Convert names list to a set once
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
end_time_optimized = time.time()
optimized_duration = end_time_optimized - start_time_optimized

print("\n--- Performance Comparison ---")
print(f"Original (Corrected) Code Execution Time: {original_duration:.6f} seconds")
print(f"Optimized Code (using set) Execution Time: {optimized_duration:.6f} seconds")
```

```
if optimized_duration < original_duration:
    print(f"The optimized code is {original_duration / optimized_duration:.2f} times faster.")
else:
    print("The optimized code is not faster in this small example (due to overhead or small data size).")
```

```
--- Original (Corrected) Code ---
Charlie is in the list
Eve is not in the list
Bob is in the list

--- Optimized Code (using set) ---
Charlie is in the list
Eve is not in the list
Bob is in the list

--- Performance Comparison ---
Original (Corrected) Code Execution Time: 0.000162 seconds
Optimized Code (using set) Execution Time: 0.000129 seconds
The optimized code is 1.26 times faster.
```

TASK3:

Prompt:

UseAItorefactoralegacyscriptwheremultiplecalculationsareembeddeddirectly inside the main code block.

BeforeCodeRefactoring:

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

```
Total Price: 295.0
Total Price: 590.0
```

AfterCodeRefactoring:

```
def calculate_total(price):
    """Calculates the total price including a fixed tax rate and prints it.

    Args:
        price (int or float): The base price of an item.
    """
    tax_rate = 0.18
    tax = price * tax_rate
    total = price + tax
    print(f"Total Price: {total}")

# Calculate totals for different prices using the new function
calculate_total(250)
calculate_total(500)
```

```
Total Price: 295.0
Total Price: 590.0
```

TASK4:

Prompt:

UseAItoidentifyandreplaceallhardcoded“magicnumbers”inthecodewithnamed constants.

BeforeCodeRefactoring:

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

AfterCodeRefactoring:

```
# Define constants
PI = 3.14159
RADIUS = 7

# Calculate and print properties of a circle using constants
print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)
```

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

TASK5:

Prompt:

UseAItoimprovereadabilitybyrenamingunclearvariablesandaddinginline comments.

BeforeCodeRefactoring:

```
a = 10
b = 20
c = a * b / 2
print(c)
```

```
100.0
```

AfterCodeRefactoring:

```
# Assign values for the base and height of the triangle
base = 10
height = 20

# Calculate the area of the triangle
# Formula for area of a triangle: (base * height) / 2
area_of_triangle = base * height / 2

# Print the calculated area
print(area_of_triangle)
```

```
100.0
```

TASK6:

Prompt:

UseAItorefactoraPythonscriptthatcontainsrepeatedif-elselogicforgrading students.

BeforeCodeRefactoring:

```
marks = 85
if marks >= 90:
  print("Grade A")
elif marks >= 75:
   print("Grade B")
else:
   print("Grade C")
marks = 72
if marks >= 90:
   print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
   print("Grade C")
```

```
Grade B
Grade C
```

AfterCodeRefactoring:

```python
def calculate_grade(marks):
    """Calculates and prints the grade based on the given marks.

    Args:
        marks (int): The score of the student.
    """
    if marks >= 90:
        print("Grade A")
    elif marks >= 75:
        print("Grade B")
    else:
        print("Grade C")

# Apply the grading function for different marks
marks = 85
calculate_grade(marks)

marks = 72
calculate_grade(marks)
```

```
Grade B
Grade C
```

TASK7:

Prompt:

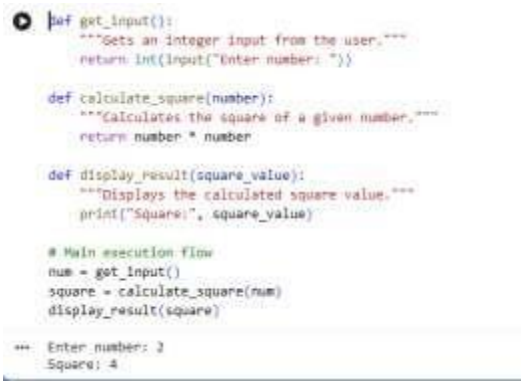
UseAItorefactorproceduralinput-processinglogicintofunctions. Before

Code Refactoring:

```python
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

```
Enter number: 2
Square: 4
```

AfterCodeRefactoring:



```python
def get_input():
    """Gets an integer input from the user."""
    return int(input("Enter number: "))

def calculate_square(number):
    """Calculates the square of a given number."""
    return number * number

def display_result(square_value):
    """Displays the calculated square value."""
    print("Square:", square_value)

# Main execution flow
num = get_input()
square = calculate_square(num)
display_result(square)
```

```
Enter number: 2
Square: 4
```

TASK8:

Prompt:

UseAItorefactorinefficientlistprocessinglogic. Before

Code Refactoring:

```
nums = [1, 2, 3, 4, 5]
squares = []
for n in nums:
    squares.append(n * n)
print(squares)
```

```
[1, 4, 9, 16, 25]
```

AfterCodeRefactoring:

```
nums = [1, 2, 3, 4, 5]
squares = [n * n for n in nums]
print(squares)
```

```
[1, 4, 9, 16, 25]
```